COUNSELLING MANAGEMENT SYSTEM

A PROJECT REPORT

Department of Information Technology

IT3711 – Summer Internship

VII SEMESTER

Submitted by

SHAGINA V. 731121205036

in partial fulfillment of the internship provided

by

Pantech eLearning Pvt Ltd.,
II Floor, Kotta Srinivasiah Charities Building,
Thanjavur Street, Near Duraisamy Subway, T.Nagar,
Chennai – 600017

JULY -2024

ACKNOWLEDGEMENT

We sincerely express our whole hearted thanks to the Pantech Solutions for their constant encouragement and moral support during the course of this internship.

We owe our sincere thanks to Mrs. Leema for finishing every essential facility for doing this Internship. We sincerely thank our mentor Ms. Vasantha for guiding us throughout the project.

Above all we are grateful to all our team members and friends for their friendly cooperation and their exhilarating company.

Sincerely,

SHAGINA V

TABLE OF CONTENTS

Chapter. No.	Title	Page
	Abstract	4
1.	**Introduction**	5
	1.1 Objective of the Project	
	1.2 Problem Statement	
	1.3 Scope of the Project	
2.	**Requirements**	6
	2.1 Functional Requirements	
	2.2 Non-Functional Requirements	
	2.3 System Specifications	
3.	**Technology Overview**	9
	3.1 Java Overview	
	3.2 Front-end Technologies	
	3.3 Database and Tools Used	
4.	**Project Directory Structure**	11
	4.1 Package Structure	
	4.2 File Descriptions	
5.	**Programming Modules**	14
	5.1 College Records Module	
	5.2 Student Records Module	
	5.3 Counselling Sections Module	
6.	**Database Connectivity and Usage**	17
	7.1 Database Design	
	7.2 ER Diagrams	
	7.3 CRUD Operations	
7.	**Web Application Snapshots**	21
	8.1 Login Page	
	8.2 Registration Pages	
	8.3 Dashboard and Other Screens	
8.	**Conclusion**	24
	9.1 Summary of Findings	
	9.2 Future Scope and Enhancements	
	References	25

ABSTRACT

Counselling Management System

The **Counselling Management System** is a comprehensive digital platform designed to modernize and enhance the student counselling process. This system addresses the inefficiencies of traditional paper-based counselling by offering a streamlined, automated solution for managing student registrations, course selections, and counselling sessions. Developed using Java and MySQL, the system provides a secure and user-friendly environment for both students and educational institutions.

Key features of the system include automated scheduling of counselling sessions, real-time updates on course availability, and secure management of sensitive student data. The system supports universities in minimizing paper usage and reducing administrative overhead, while also improving the accuracy and accessibility of counselling services. Through its intuitive interface, students can easily register for courses and schedule appointments, while counsellors can efficiently manage their sessions and provide guidance.

Overall, the **Counselling Management System** aims to enhance the efficiency, effectiveness, and user experience of educational counselling services, providing a scalable and reliable tool for modern educational institutions.

CHAPTER 1

INTRODUCTION

1.1 Objective of the Project

The primary objective of the **Counselling Management System** is to provide an efficient and streamlined process for students seeking guidance regarding educational policies and college registration. This system is designed to digitize and automate the counselling process, making it easier for universities to manage their records and reduce paper usage, while also offering students comprehensive support throughout their academic journey.

The system facilitates interactions between students, counsellors, and educational institutions by providing a platform where students can register for counselling sessions, get advice on course selections, and receive guidance on the college admission process. Additionally, it provides universities with a means to store and manage data related to student records, course offerings, and counsellors, thereby improving data accessibility and accuracy.

1.2 Problem Statement

Traditionally, the counselling process involves significant paperwork, manual management of student data, and often leads to inefficiencies, delays, and errors in guiding students through their educational journey. Students face difficulties in receiving timely and accurate information regarding available courses and registration requirements. Universities also struggle to manage the vast amount of paperwork generated during the process, which increases the risk of data loss or mismanagement.

1.3 Scope of the Project

This **Counselling Management System** provides a holistic solution by automating various aspects of the counselling process. The system allows universities to digitally store and manage student records, provide detailed information about courses and registration processes, and offer a streamlined communication channel between students and counsellors. The system is composed of three core modules: College Records Management, Student Records Management, and Counselling Sessions Management.

The scope includes automating the student registration process, providing guidance on course selections, improving data management for universities, and ultimately reducing the reliance on paper-based systems.

CHAPTER 2

REQUIREMENTS

2.1 Functional Requirements

Functional requirements define the specific functionalities that the system must provide to meet the project's objectives. The key functional requirements for the **Counselling Management System** are:

- **User Authentication and Authorization:**
  - The system must allow users (students, counsellors, and administrators) to register and log in securely.
  - Different roles (students, counsellors, administrators) should have access to specific functionalities based on their privileges.
- **Student Records Management:**
  - The system should allow students to create and update their profiles with personal information, educational background, and preferred course selections.
  - Students should be able to view their counselling history and registered courses.
- **Counselling Session Management:**
  - Students must be able to schedule counselling sessions based on counsellor availability.
  - Counsellors should be able to view their scheduled sessions, manage appointments, and provide guidance to students.
  - The system should support virtual or in-person counselling sessions and keep a record of all sessions.
- **College Records Management:**
  - Universities should be able to maintain a comprehensive database of their course offerings, faculty information, and admissions policies.
  - The system should allow administrators to update, delete, or add new college records as needed.
- **Course Registration Support:**

    - The system must guide students through the process of course registration by providing real-time information on available courses, registration deadlines, and eligibility criteria.
    - Notifications should be sent to students regarding important deadlines and updates.
- **Data Storage and Retrieval:**
    - All data related to students, counsellors, and colleges should be securely stored in a centralized database.
    - The system should provide fast and accurate retrieval of data for students and universities.
- **Notifications and Alerts:**
    - The system should send automated email notifications and alerts to students about upcoming counselling sessions, registration deadlines, and any changes in course offerings.

2.2 Non-Functional Requirements

Non-functional requirements focus on the system's performance, security, and usability. These are critical to ensuring that the system operates efficiently and effectively.

- **Performance:**
    - The system must be able to handle multiple users concurrently without performance degradation.
    - All operations, including database transactions and user interactions, should complete within acceptable time limits (e.g., under 2 seconds).
- **Security:**
    - The system must provide secure access to authorized users through strong authentication methods (e.g., encrypted passwords, SSL).
    - User data and records should be protected from unauthorized access, both during storage and transmission.
- **Scalability:**
    - The system should be scalable to accommodate the growing number of students and counsellors using the platform over time.

    - It should be flexible enough to add new features and modules without disrupting existing services.
- **Usability:**
    - The user interface must be intuitive and easy to navigate for all types of users (students, counsellors, and administrators).
    - Minimal training should be required for users to understand how to utilize the system's features.

2.3 System Specifications

The system must meet the following technical specifications to ensure smooth operation:

- **Operating System:**
    - The application should be deployable on both Windows and Linux servers.
- **Programming Languages:**
    - **Backend:** Java for server-side logic.
    - **Frontend:** HTML, CSS, JavaScript for user interface design.
- **Database:**
    - MySQL (or any other relational database) for storing student records, college data, and session logs.
- **Framework:**
    - Spring Boot for backend development to manage application structure and dependencies.
- **Server:**
    - Apache Tomcat for hosting the web application.
- **Tools and Libraries:**
    - Hibernate for ORM (Object-Relational Mapping) to manage database interactions.
    - Maven or Gradle for project build and dependency management.

CHAPTER 3

TECHNOLOGY OVERVIEW

This section provides an overview of the technologies used in the development of the **Counselling Management System**, explaining their roles and contributions to the project.

3.1 Java Overview

Java is the cornerstone of the back-end development for the **Counselling Management System**. As a powerful, platform-independent language, Java offers several advantages:

- **Object-Oriented Programming**: Facilitates modular, reusable, and maintainable code.
- **Robust Standard Libraries**: Provides a wide range of built-in functionalities for data handling, networking, and security.
- **Scalability**: Supports large-scale applications with high performance and stability.
- **Security**: Offers strong security features to protect sensitive user data and ensure safe operations.

Java's ecosystem, including frameworks and tools, enables efficient development, integration, and deployment of the system, ensuring that it meets the required performance and security standards.

3.2 Front-end Technologies

The user interface of the **Counselling Management System** is crafted using modern web technologies:

- **HTML (HyperText Markup Language)**: Structures the content of web pages, including forms, tables, and navigation elements.
- **CSS (Cascading Style Sheets)**: Defines the visual style and layout of the web pages, ensuring a consistent and aesthetically pleasing design.
- **JavaScript**: Enhances the interactivity and functionality of the web pages, enabling dynamic content updates, form validation, and user interactions.

Together, these front-end technologies create an engaging and intuitive user experience, allowing students, counsellors, and administrators to interact with the system efficiently.

3.3 Database and Tools Used

The **Counselling Management System** employs **MySQL** for database management. Key aspects include:

- **MySQL**: A widely-used relational database management system that supports complex queries, data integrity, and scalability. It efficiently handles the storage and retrieval of data related to students, courses, and counselling sessions.
- **JDBC (Java Database Connectivity)**: Acts as a bridge between the Java application and the MySQL database, providing a standard API for executing SQL queries and updates. JDBC ensures secure and reliable database interactions.
- **SQL (Structured Query Language)**: Used for designing, querying, and managing the database schema. SQL enables efficient data operations such as inserting, updating, deleting, and retrieving information from the database.

Additional tools and technologies include:

- **Database Management Tools**: Tools like MySQL Workbench are used for database design, management, and query execution, providing a graphical interface for database administration tasks.

These technologies and tools collectively ensure that the system's data management is efficient, secure, and scalable, supporting the overall functionality of the **Counselling Management System**.

CHAPTER 4

PROJECT DIRECTORY STRUCTURE

4.1 src/main/java

Contains core Java source files organized by MVC pattern:

- **controller/**: Manages HTTP requests (e.g., StudentController.java, CounsellorController.java).
- **model/**: Defines data structures (e.g., Student.java, CounsellingSession.java).
- **repository/**: Handles database operations (e.g., StudentRepository.java, CounsellingSessionRepository.java).
- **service/**: Contains business logic (e.g., StudentService.java, CounsellingService.java).
- **CounsellingManagementSystemApplication.java**: Main application entry point.

4.2 src/main/resources

Includes static resources and configuration:

- **static/**: Contains CSS, JavaScript, and images.
- **templates/**: HTML templates for front-end (e.g., login.html, dashboard.html).
- **application.properties**: Configuration settings (e.g., database connection).
- **data.sql**: Script for initial data population.

4.3 src/test/java

Holds unit and integration tests for the application:

- **controller/**: Tests for controller functionality.

4.4 pom.xml (or build.gradle)

Manages build and dependencies:

- **pom.xml**: Maven configuration.

- **build.gradle**: Gradle configuration.

4.5 docs/

Contains project documentation:

- **design_diagrams/**: UML and ER diagrams.
- **user_manual.pdf**: Instructions for using the system.
- **project_report.pdf**: Comprehensive project report.

```
counselling-management-system/
│
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/
│   │   │       └── counselling/
│   │   │           ├── controller/
│   │   │           ├── model/
│   │   │           ├── repository/
│   │   │           ├── service/
│   │   │           └── CounsellingManagementSystemApplication.java
│   │   └── resources/
│   │       ├── static/
│   │       │   ├── css/
│   │       │   ├── js/
│   │       │   └── images/
│   │       ├── templates/
│   │       │   ├── login.html
```

```
│   │       │   ├── dashboard.html
│   │       │   ├── student_profile.html
│   │       │   └── counselling_sessions.html
│   │       ├── application.properties
│   │       └── data.sql
│   └── test/
│       └── java/
│           └── com/
│               └── counselling/
│                   └── controller/
├── pom.xml (or build.gradle)
├── README.md
└── docs/
    ├── design_diagrams/
    ├── user_manual.pdf
    └── project_report.pdf
```

CHAPTER 5

PROGRAMMING MODULES

The **Counselling Management System** is divided into three primary modules: **College Records Module**, **Student Records Module**, and **Counselling Sections Module**. Each module is implemented using Java, following the MVC (Model-View-Controller) architecture. The code is structured for scalability, modularity, and reusability.

5.1 College Records Module

Key Functionalities:

- **College Registration**: This feature allows administrators to register new colleges or universities. The code uses form data validation and persistence logic, leveraging a repository class (CollegeRepository.java) to interact with the database.

```
@PostMapping("/registerCollege")
public String registerCollege(@ModelAttribute College college) {
    collegeService.save(college);
    return "redirect:/colleges";
}
```

- **Course Management**: Enables administrators to create, update, or delete courses. This is handled through a controller method that maps HTTP requests to service layer logic. A repository interface (CourseRepository.java) facilitates CRUD operations.

```
@PutMapping("/updateCourse/{id}")
public String updateCourse(@PathVariable Long id, @ModelAttribute Course course) {
    courseService.updateCourse(id, course);
    return "redirect:/courses";
}
```

- **University Information**: The module retrieves data from the database using methods such as findAll() to list available universities and their corresponding details.

```
public List<College> getAllColleges() {
    return collegeRepository.findAll();
}
```

5.2 Student Records Module

Key Functionalities:

- **Student Registration**: This functionality includes form validation and data persistence to store new student records in the database. The service class (StudentService.java) uses methods like save() for storing student data.

```
@PostMapping("/registerStudent")
public String registerStudent(@ModelAttribute Student student) {
    studentService.save(student);
    return "redirect:/students";
}
```

- **Course Enrollment**: Handles course registrations for students. The StudentController.java receives the student ID and course ID and triggers a service layer method to update the enrollment records.

```
@PostMapping("/enroll")
public String enrollCourse(@RequestParam Long studentId, @RequestParam Long courseId)
    enrollmentService.enrollStudentInCourse(studentId, courseId);
    return "redirect:/courses";
}
```

- **Profile Management**: Allows students to edit and update their profile. A method within StudentService.java retrieves and updates data from the database.

```
public void updateProfile(Long studentId, Student updatedStudent) {
    Student student = studentRepository.findById(studentId).orElseThrow();
    student.setName(updatedStudent.getName());
    student.setEmail(updatedStudent.getEmail());
    studentRepository.save(student);
}
```

5.3 Counselling Sections Module

Key Functionalities:

- **Session Scheduling**: This feature allows students to book counselling sessions. The controller method handles requests and interacts with the database through a repository (CounsellingSessionRepository.java).

```
@PostMapping("/scheduleSession")
public String scheduleSession(@ModelAttribute CounsellingSession session) {
    sessionService.scheduleSession(session);
    return "redirect:/sessions";
}
```

- **Session Tracking**: Tracks the status of scheduled sessions. The service layer retrieves information from the database and updates session details as necessary.

```
public List<CounsellingSession> getSessionsByStudent(Long studentId) {
    return sessionRepository.findByStudentId(studentId);
}
```

- **Counsellor Management**: This feature allows administrators to manage counsellor profiles, including their availability and areas of expertise. It uses similar CRUD operations, with entities mapped to the database.

```
public void updateCounsellor(Long counsellorId, Counsellor updatedCounsellor) {
    Counsellor counsellor = counsellorRepository.findById(counsellorId).orElseThrow();
    counsellor.setAvailability(updatedCounsellor.getAvailability());
    counsellorRepository.save(counsellor);
}
```

Each module is carefully designed to maintain clean separation of concerns between the layers, ensuring scalability and ease of maintenance. The repository layer uses JPA to abstract database operations, while the service layer handles business logic, and the controller layer manages user interaction and request handling.

CHAPTER 6

DATABASE CONNECTIVITY AND USAGE

The **Counselling Management System** relies on a relational database to manage and persist data related to students, counsellors, courses, and counselling sessions. This section covers the database design, the Entity-Relationship (ER) diagrams, and the CRUD (Create, Read, Update, Delete) operations performed by the system.

6.1 Database Design

The database design for the Counselling Management System involves creating tables and defining relationships to effectively manage the data. The primary entities and their attributes are:

- **Student**
  - student_id: Unique identifier for each student
  - name: Full name of the student
  - email: Email address of the student
  - profile: Additional profile information
  - registered_courses: Courses the student is registered for
- **Course**
  - course_id: Unique identifier for each course
  - name: Name of the course
  - university_id: Identifier for the university offering the course
  - description: Description of the course
- **Counsellor**
  - counsellor_id: Unique identifier for each counsellor
  - name: Full name of the counsellor
  - specialization: Area of expertise
  - availability: Counsellor's availability for sessions

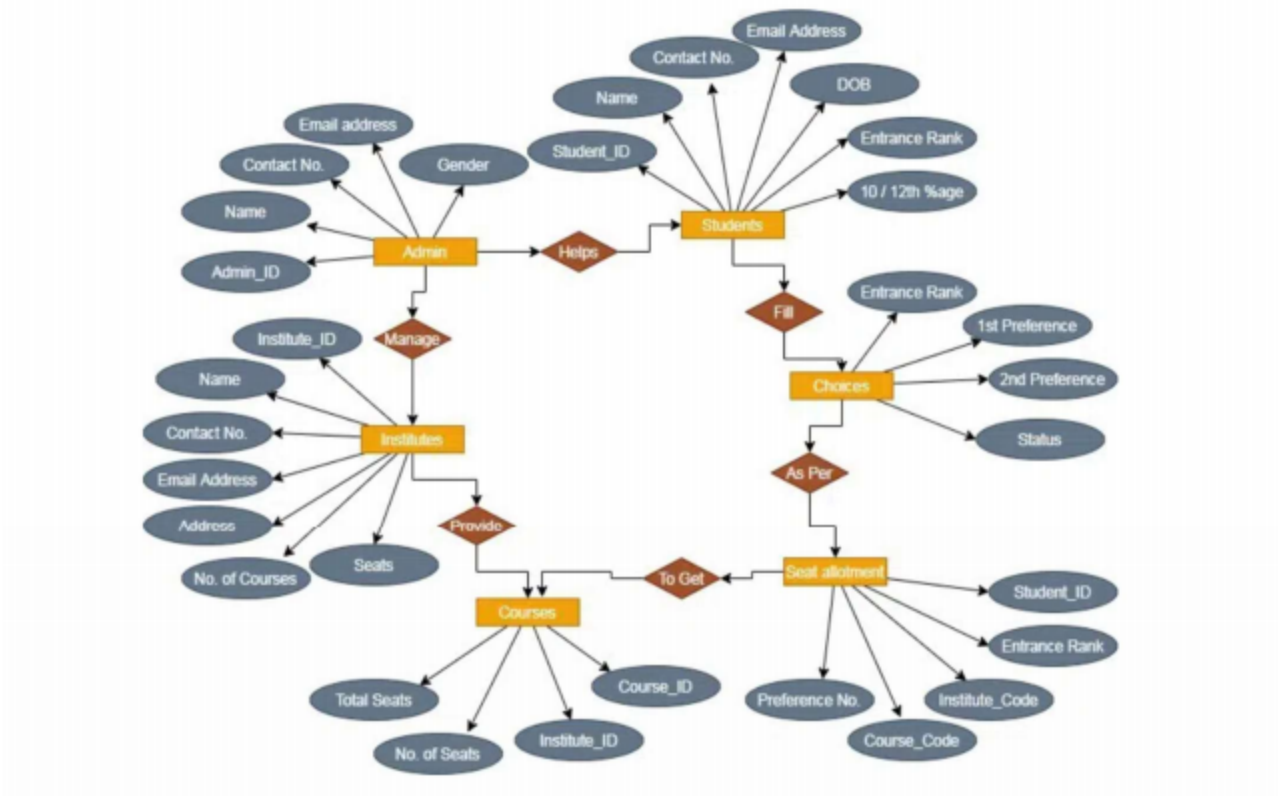
- **CounsellingSession**
  - session_id: Unique identifier for each counselling session
  - student_id: Identifier for the student participating in the session
  - counsellor_id: Identifier for the counsellor conducting the session
  - date: Date and time of the session
  - status: Current status of the session (e.g., scheduled, completed)
- **University**
  - university_id: Unique identifier for each university
  - name: Name of the university
  - location: Location of the university
  - admission_policies: Admission policies of the university

6.2 ER Diagrams

The **Entity-Relationship (ER) Diagrams** provide a visual representation of the database schema and the relationships between different entities. Key relationships include:

- **One-to-Many Relationship**:
  - **University to Course**: One university can offer multiple courses.
  - **Student to CounsellingSession**: One student can have multiple counselling sessions.
- **Many-to-One Relationship**:
  - **Course to University**: Each course belongs to one university.
  - **CounsellingSession to Counsellor**: Each session is conducted by one counsellor.
  - **CounsellingSession to Student**: Each session involves one student.

The ER diagrams visually illustrate these relationships and help in understanding how the data is interconnected.



6.3 CRUD Operations

The system performs various CRUD (Create, Read, Update, Delete) operations to manage the data. Here's a brief overview of how these operations are implemented:

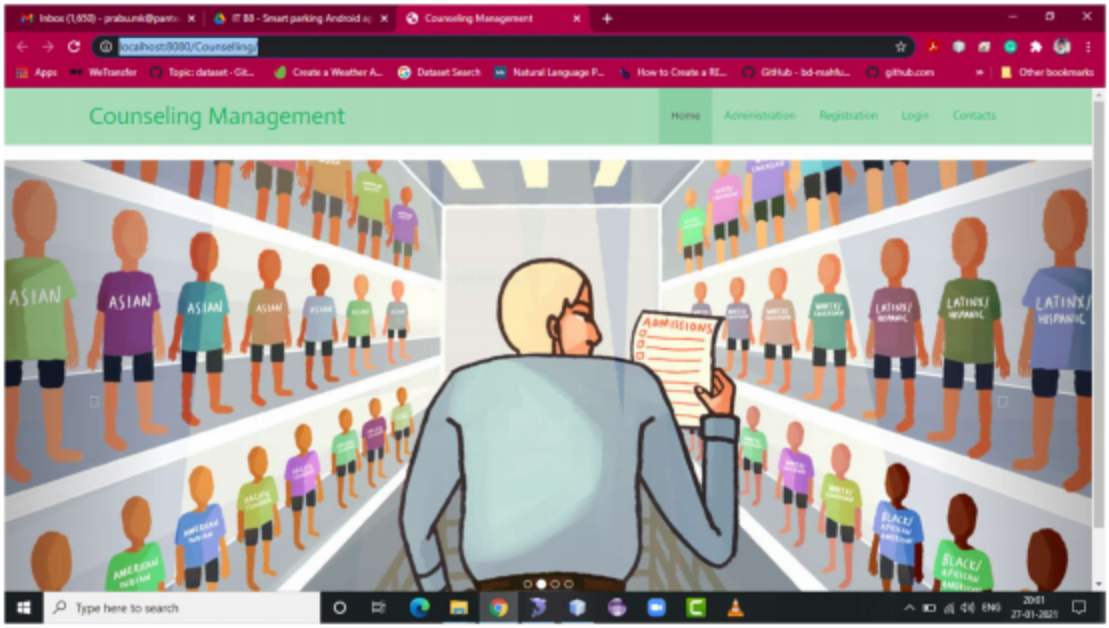
- **Create**:
  - **Add Student**: New student records are created in the database when a student registers.
  - **Add Course**: New courses are added by university administrators.
- **Read**:
  - **View Student Profile**: Students and administrators can view detailed student profiles.
  - **List Courses**: Users can view available courses offered by universities.

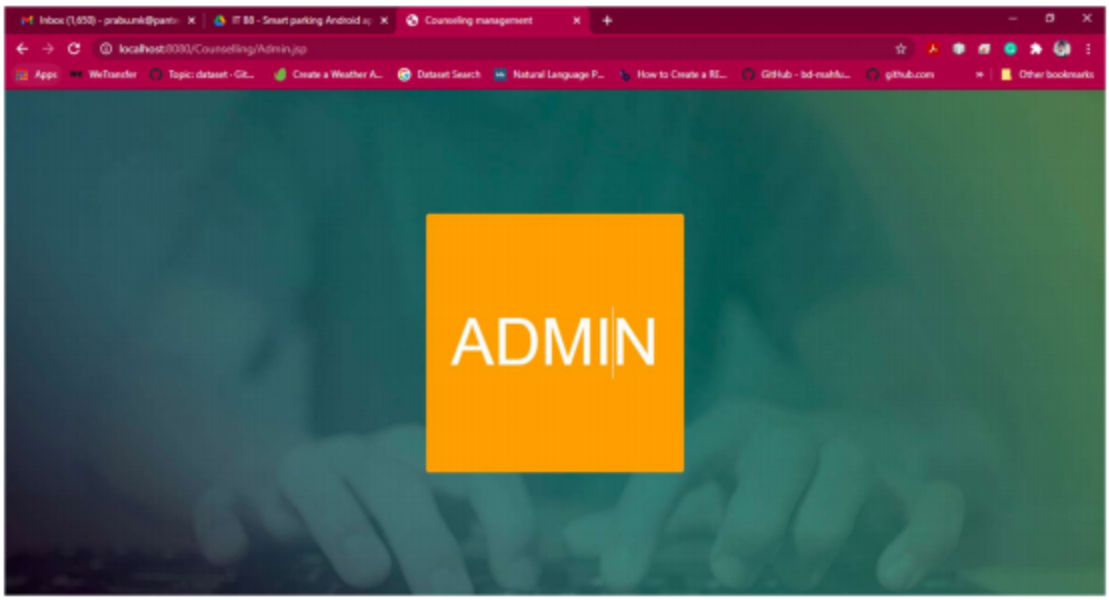
- **Update**:
  - **Update Student Information**: Students can update their profile details and course registrations.
  - **Modify Course Details**: Universities can update course information as needed.
- **Delete**:
  - **Remove Student**: Student records can be deleted if a student is removed from the system.
  - **Delete Course**: Courses can be removed if they are no longer offered.

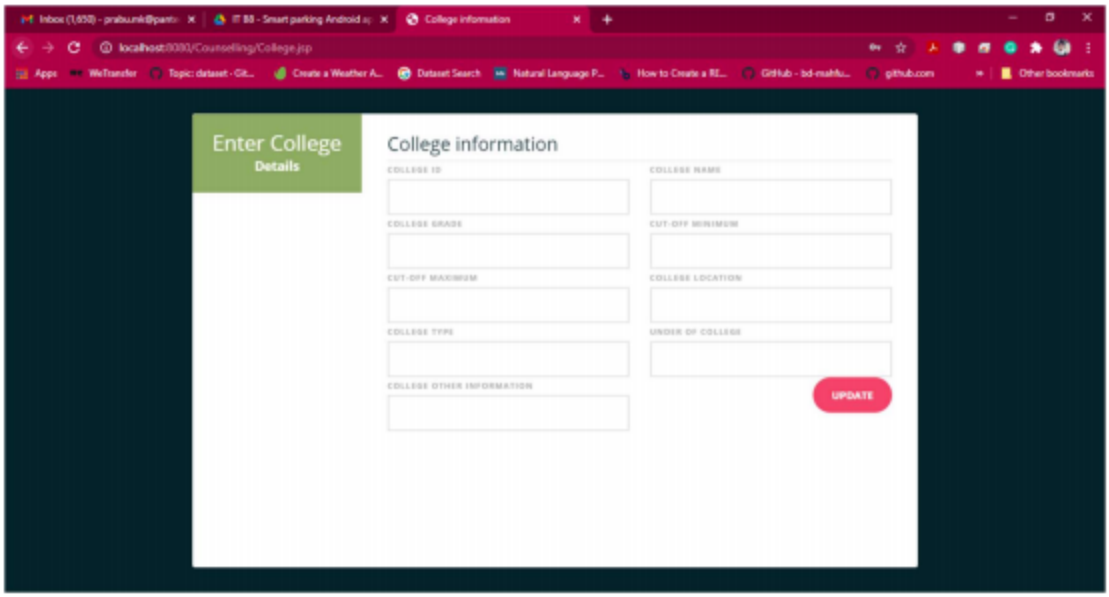
CRUD operations are implemented using Spring Data JPA repositories, which provide an abstraction layer over database interactions, making it easier to perform these operations through simple method calls.

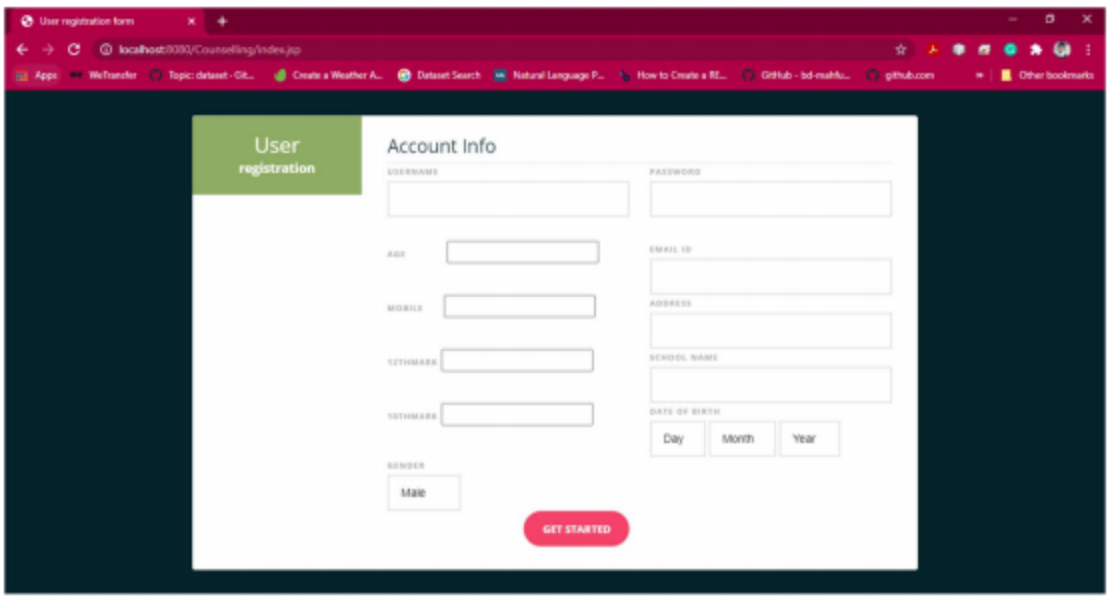
CHAPTER 7

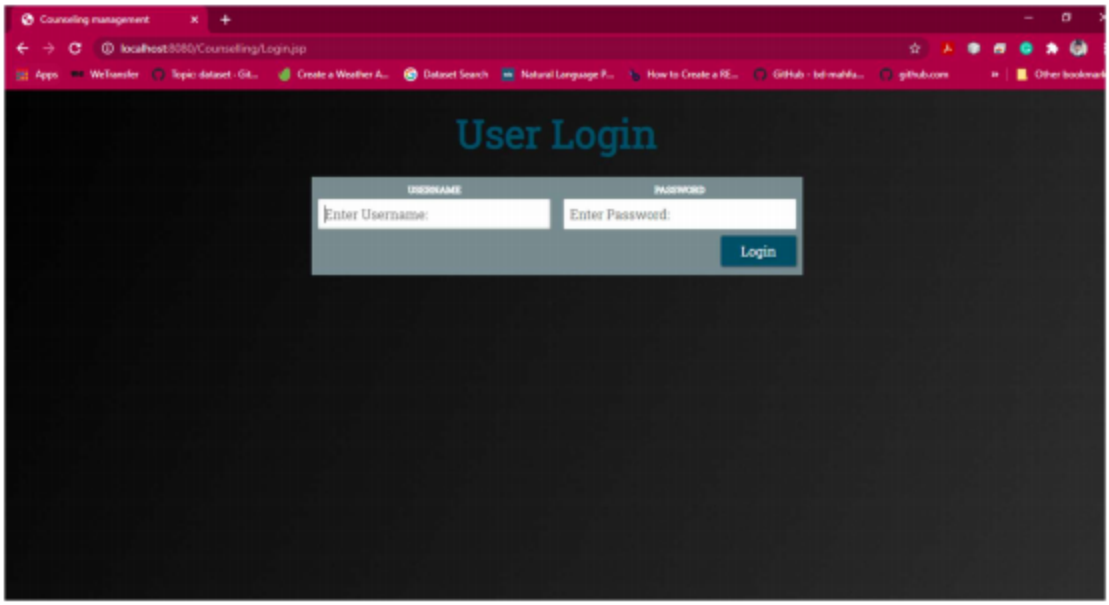
WEB APPLICATION SNAPSHOTS

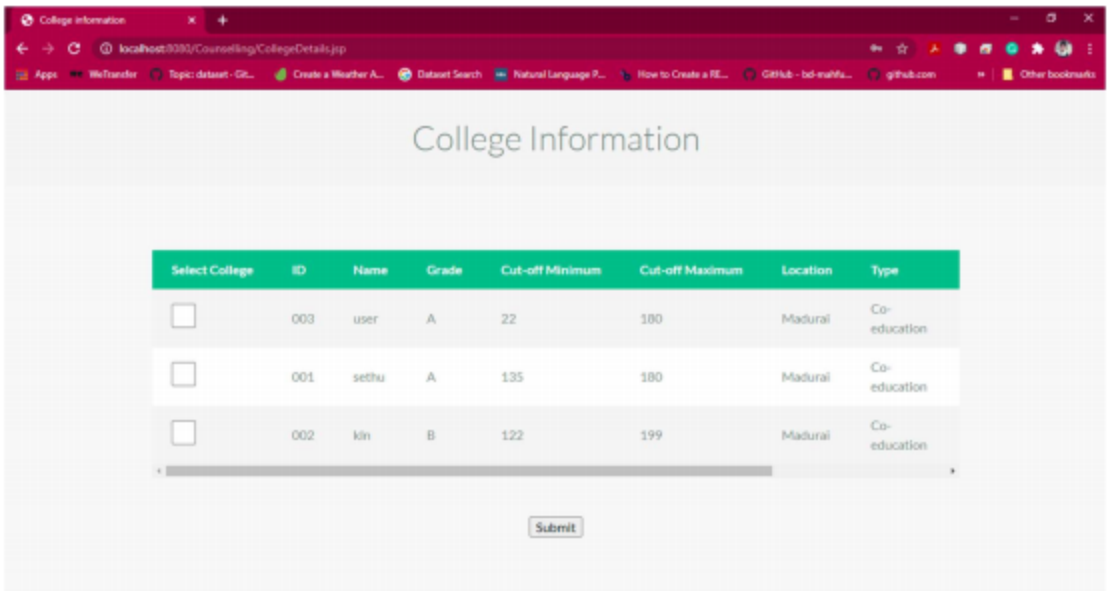





Enter College
Details
College information
COLLEGE ID
COLLEGE NAME
COLLEGE GRADE
CUT-OFF MINIMUM
CUT-OFF MAXIMUM
COLLEGE LOCATION
COLLEGE TYPE
UNDER OF COLLEGE
COLLEGE OTHER INFORMATION
UPDATE


User
registration
Account Info
USERNAME
PASSWORD
AGE
EMAIL ID
MOBILE
ADDRESS
12THMARK
SCHOOL NAME
10THMARK
DATE OF BIRTH
Day
Month
Year
GENDER
Male
GET STARTED


Counseling management
localhost:8080/Counselling/Login.jsp
User Login
USERNAME
PASSWORD
Enter Username:
Enter Password:
Login


College information
localhost:8080/Counselling/CollegeDetails.jsp
College Information
Select College
ID
Name
Grade
Cut-off Minimum
Cut-off Maximum
Location
Type
003 user A 22 180 Madurai Co-education
001 sethu A 135 180 Madurai Co-education
002 kin B 122 199 Madurai Co-education
Submit

CHAPTER 8

CONCLUSION

8.1 Summary of the Project

The **Counselling Management System** was developed to digitize and streamline the counselling process for students and educational institutions. By providing functionalities such as course registration, counselling session scheduling, and guidance management, the system reduced paperwork and improved the overall efficiency of counselling services. Built using Java Spring Boot and MySQL, the system features a modular design that ensures scalability, security, and ease of maintenance.

8.2 Achievements and Learnings

Key achievements include:

- **Modular Design**: A three-tier architecture facilitated ease of maintenance and scalability.
- **Automation**: The system automated scheduling and session management, reducing manual work.
- **Data Security**: Robust security measures were implemented to protect sensitive data.

From this project, we gained skills in Java development, Spring Boot, database management, and system architecture design, while also enhancing our understanding of project planning and execution.

8.3 Future Enhancements

Potential future improvements include:

- **AI-Driven Recommendations**: Integrating AI to provide personalized guidance.
- **Mobile App Development**: Creating a mobile version for wider accessibility.
- **Real-Time Communication**: Adding chat or video conferencing features for direct interaction between students and counsellors.

CHAPTER 9

REFERENCES

1. **Spring Framework Documentation -** "Spring Boot Reference Documentation", Pivotal Software, Inc. Available at: https://docs.spring.io/spring-boot/docs/current/reference/html/
2. **Hibernate ORM Documentation -** "Hibernate ORM User Guide", Red Hat. Available at: https://hibernate.org/orm/documentation/
3. **MySQL Documentation -** "MySQL 8.0 Reference Manual", Oracle Corporation. Available at: https://dev.mysql.com/doc/refman/8.0/en/
4. **Thymeleaf Documentation -** "Thymeleaf Official Documentation", Thymeleaf. Available at: https://www.thymeleaf.org/documentation.html
5. **HTML5 and CSS3: Visual QuickStart Guide** by Elizabeth Castro and Bruce Hyslop, 8th Edition, Peachpit Press, 2013.
6. **Effective Java** by Joshua Bloch, 3rd Edition, Addison-Wesley, 2018.
7. **Java: The Complete Reference** by Herbert Schildt, 11th Edition, McGraw-Hill Education, 2018.
8. **Head First Design Patterns** by Eric Freeman and Elisabeth Robson, O'Reilly Media, 2004.
9. **Designing Data-Intensive Applications** by Martin Kleppmann, O'Reilly Media, 2017.
10. **RESTful Web Services** by Leonard Richardson and Sam Ruby, O'Reilly Media, 2007.